THLPro — Documentação da API

A API do THLPro é documentada automaticamente em **Swagger/OpenAPI** no endpoint `/docs` (e `openapi.json` na raiz do projeto). Este documento resume os grupos de endpoints e as regras principais de uso.

- Base: `http://<servidor>/api` (no projeto, o nginx roteia `/api` → API; em dev, o Vite faz proxy `/api` → `localhost:8000`).
- Documentação interativa: `/docs` · Especificação: `/openapi.json`.

Autenticação

Método	Rota	Descrição
POST	`/auth/login`	Login (email+senha) → `{access_token, user}`
POST	`/auth/logout`	Logout (invalida token)
GET	`/auth/me`	Dados do usuário autenticado
POST	`/auth/refresh`	Renovação de token (se habilitado)

- Token: `Authorization: Bearer <access_token>` (JWT, expiração configurável, default 8h).
- Perfis: `admin` (todas as rotas) e `user` (dados de mercado e operação).

Usuários (admin)

Método	Rota	Descrição
GET/POST	`/users`	Listar/criar usuários
GET/PUT/DELETE	`/users/{user_id}`	Detalhar/atualizar/desativar usuário
GET/PUT	`/users/me/preferences`	Preferências individuais (ativo, strike, nº vencimentos, CALL/PUT, auto-refresh, intervalo, tema) — GET cria default se ausente; PUT aceita body parcial

Ativos

Método	Rota	Descrição
GET/POST	`/assets`	Listar (filtro `enabled`) / criar ativo (admin)
GET/PUT/DELETE	`/assets/{id}`	Detalhar / atualizar / desativar (admin)
GET	`/assets/{symbol}/strikes`	**Grade de strikes**: faixa ±% (configurado) em torno do preço spot atual, com TODOS os strikes de séries ativas dentro da faixa (sem filtro 0/5), ordenados
GET	`/assets/{symbol}/strikes/range`	Faixa (range_low/range_high) + spot
GET	`/assets/{symbol}/spot`	Última cotação do ativo-base
POST	`/assets/quotes`	Recebe cotação do ativo-base (cliente/user)

Campos do ativo: `symbol, name, asset_type, market, enabled, tick_min, strike_interval, lower_range_percent, upper_range_percent, rtd_identifier_pattern, source`.

Opções

Método	Rota	Descrição
GET	`/options`	Lista séries (filtros asset/type/vencimento/ativo)
GET	`/options/search?q=`	Busca por código
GET	`/options/by-strike?asset=&strike=&expiration_count=6&types=`	Séries por strike + vencimentos (usa tolerância configurada; `matched_strike` quando a série é próxima e não exata)
GET	`/options/by-expiration?asset=&expiration=`	Séries de um vencimento
POST	`/options/synchronize`	**Sincronização B3** (admin): `{source: "b3"|"upload", asset_id?}` (+ upload multipart opcional) → `{created, updated, deactivated, errors}`

Cotações

Método	Rota	Descrição
POST	`/quotes`	Envia lista de cotações (cliente autenticado por API key **ou** usuário) → `{accepted, rejected:[{index, reason}]}`
POST	`/quotes/client`	Idem, autenticação explícita de cliente
GET	`/quotes/latest?asset=&option_symbol=`	Última cotação aceita
GET	`/quotes/history?asset=&option_symbol=&limit=`	Histórico recente
GET	`/quotes`	Consulta com filtros (asset/symbol/intervalo)

Payload de cotação (cliente → servidor):

```
{
  "asset": "BOVA11",
  "option_symbol": "BOVAX185",
  "option_type": "CALL",
  "strike": 185.0,
  "expiration_date": "2026-10-16",
  "last_price": 3.75,
  "bid": 3.33,
  "ask": 3.61,
  "source": "Profit RTD",
  "timestamp": "2026-10-08T10:25:30-03:00"
}
```

Regras do servidor: dado sem `timestamp` é rejeitado (400/422); dado com timestamp ≤ último aceito para a mesma série é rejeitado (**anti-rollback**); entre dois clientes, vence o timestamp mais recente e válido (skew máximo configurável), depois a sequência de recebimento; `premium` = último preço (padrão da planilha).

Clientes locais (.exe/RTD)

Método	Rota	Descrição
POST	`/clients/register`	Cadastra cliente (auth usuário): `{client_name, computer_name}` → `{client_id, api_key}` (api_key exibida **uma única vez**)
POST	`/clients/heartbeat`	Batimento (auth API key) — mantém `connected` e `last_seen_at`
POST	`/clients/quotes`	Envia lotes de cotações (auth API key)
GET	`/clients/config`	Config para o cliente: ativos habilitados + séries ativas com `rtd_symbol` (ex.: `BOVAJ40_B_0`) — o cliente assina esses tópicos no Profit
GET	`/clients/status`	Diagnóstico (admin): conectados, último heartbeat, última atualização, erros, contadores
POST	`/clients/{id}/disconnect` / `reconnect`	Forçar desconexão / reconexão (admin)

Administração (admin)

Método	Rota	Descrição
GET	`/admin/logs?event_type=&limit=`	Logs do sistema (login, RTD, erros, rejeições)
GET	`/admin/settings`	Configurações globais
POST	`/admin/settings/{key}`	Atualiza configuração (`strike_tolerance`, `data_max_age_seconds`, `simulation_mode`, ...)
POST	`/admin/simulation/generate`	Gera dados **simulados** de teste (somente com `simulation_mode=true`; origem `SIMULATED`, visível nas telas)

WebSocket

Rota	Descrição
`/ws/market-data?token=<jwt>`	Broadcast de cotações de opções (`type:"quote"`) e do ativo-base (`type:"asset_quote"`) para usuários autenticados; envia snapshot ao conectar
`/ws/client-status?token=<jwt admin>`	Eventos de conexão/desconexão dos clientes locais

Saúde

Método	Rota	Descrição
GET	`/health`	`{status, version, time_utc, database?}`

Consulte `/docs` (Swagger) para contratos completos de request/response de cada endpoint.